

```
1 #include <stdio.h>
2
3 int add(int a, int b);
4 float average(int a, int b, int c);
5 int factorial(int n);
6 void swap(int a, int b);
7 int maximum(int a, int b);
8 int fibonacci(int n);
9
10 int main()
11 {
12     int num1 = 10;
13     int num2 = 20;
14
15     int result;
16
17     result = add(num1);
18
19     // Error: Function add() requires two arguments but only one is provided.
20
21     // Solution: result = add(num1, num2);
22
23     float avg;
24
25     avg = average(70,80);
26
27     // Error: Function average() requires three arguments but only two are provided.
28
29     // Solution: avg = average(70, 80, 90);
30
31 }
```

```
25  printf("Average = %.2f\n", avg);
26
27  int fact;
28
29  fact = factorial(5);
30
31  printf("Factorial = %d\n", fact);
32
33  printf("\n");
34
35  int x = 100;
36  int y = 200;
37
38  printf("Before Swap\n");
39  printf("%d %d\n", x, y);
40
41  swap(x,y);
    // Warning: swap() uses pass-by-value; x and y will not actually be swapped.
    // Solution: Pass addresses and use pointers: swap(&x, &y);
42
43  printf("After Swap\n");
44  printf("%d %d\n", x, y);
45
46  printf("\n");
47
48  int max;
49
50  max = maximum(15,25);
```

```
51
52  printf("Maximum = %d\n", max);
53
54  printf("\n");
55
56  int fib;
57
58  fib = fibonacci(10);
59
60  printf("Fibonacci = %d\n", fib);
61
62  printf("\n");
63
64  int marks = 90;
65
66  calculateGrade(marks);
    // Error: calculateGrade() is called but no declaration or definition exists.
    // Solution: Define calculateGrade() or replace with a valid function call.
67
68  printf("\n");
69
70  int area;
71
72  area = circleArea(5);
    // Error: circleArea() is called but no declaration or definition exists.
    // Solution: Define circleArea() before use.
73
74  printf("%d\n", area);
```

```
75
76  printf("\n");
77
78  int total;
79
80  total = sumArray();
    // Error: sumArray() is called but no declaration or definition exists.
    // Solution: Define sumArray() before use.
81
82  printf("%d\n", total);
83
84  printf("\n");
85
86  displayResult();
    // Error: displayResult() is called but no declaration or definition exists.
    // Solution: Define displayResult() before use.
87
88  printf("\n");
89
90  return 0;
91 }
92
93 int add(int a, int b)
94 {
95     return a + b;
96 }
97
98 float average(int a, int b, int c)
```

```
99 {
100     return (a + b + c) / 3;
    // Warning: Integer division occurs before conversion to float.
    // Solution: return (a + b + c) / 3.0f;
101 }
102
103 int factorial(int n)
104 {
105     if(n == 0)
106         return 0;
    // Error: Factorial of 0 should be 1.
    // Solution: return 1;
107
108     return n * factorial(n - 1);
109 }
110
111 void swap(int a, int b)
112 {
113     int temp;
114
115     temp = a;
116     a = b;
117     b = temp;
118 }
119
120 int maximum(int a, int b)
121 {
122     if(a > b)
```

```

123     return b;

        // Error: Returns smaller value when a > b.

        // Solution: return a;
124     else
125         return a;

        // Error: Returns smaller value when a <= b.

        // Solution: return b;
126 }
127
128 int fibonacci(int n)
129 {
130     if(n == 0)
131         return 0;
132
133     if(n == 1)
134         return 1;
135
136     return fibonacci(n) + fibonacci(n-1);

        // Error: Causes infinite recursion because fibonacci(n) calls itself with same
        // argument.

        // Solution: return fibonacci(n - 1) + fibonacci(n - 2);
137 }
138
139 void printMessage()
140 {
141     printf("Welcome\n");
142 }
143

```

```
144 int calculatePercentage(int marks, int total)
145 {
146     return marks / total * 100;
        // Warning: Integer division may truncate results before multiplication.
        // Solution: return (marks * 100) / total; or use floating-point arithmetic.
147 }
148
149 float simpleInterest(float p, float r, float t)
150 {
151     float si;
152
153     si = (p * r * t) / 100;
154
155     return;
        // Error: Non-void function returns no value.
        // Solution: return si;
156 }
157
158 void displayMenu()
159 {
160     printf("1. Add\n");
161     printf("2. Subtract\n");
162     printf("3. Exit\n");
163 }
164
165 int power(int base, int exp)
166 {
167     int result = 1;
```

```
168
169  for(int i=1; i<exp; i++)
170  {
171      result *= base;
172  }
173
174  return result;

    // Error: Loop executes one time fewer than required.

    // Solution: for(int i = 0; i < exp; i++) or for(int i = 1; i <= exp; i++)
175 }
176
177 int reverseNumber(int n)
178 {
179     int rev = 0;
180
181     while(n > 0)
182     {
183         rev = rev * 10 + n % 10;
184         n = n / 10;
185     }
186 }

    // Error: Non-void function does not return a value.

    // Solution: return rev;
187
188 int countDigits(int n)
189 {
190     int count = 0;
191
```



```
192 while(n > 0)
193 {
194     count++;
195     n = n / 10;
196 }
197
198 return count;
199}
200
201 int isPrime(int n)
202 {
203     for(int i=2; i<n; i++)
204     {
205         if(n % i == 0)
206             return 1;
207         // Error: Returns 1 when number is not prime.
208         // Solution: return 0;
209     }
210 }
211
212 void updateValue(int x)
213 {
214     x = 500;
215     // Warning: Modification affects only local copy due to pass-by-value.
```

// Solution: Use a pointer parameter if caller's value should be modified.

215 }

216

217 int findMinimum(int a, int b)

218 {

219 if(a < b)

220 return a;

221 else

222 return b;

223 }

224

225 void printArray(int arr[], int size)

226 {

227 for(int i=0; i<=size; i++)

 // Error: Accesses arr[size], causing out-of-bounds access.

 // Solution: for(int i = 0; i < size; i++)

228 {

229 printf("%d ", arr[i]);

230 }

231 }

232

233 int recursiveDemo(int n)

234 {

235 return recursiveDemo(n+1);

 // Error: Infinite recursion with no terminating condition.

 // Solution: Add a base case before recursive call.

236 }

237

```
238 void greet()
239 {
240     printf("Hello Students\n");
241 }
242
243 float rectangleArea(float length, float width)
244 {
245     return length + width;
246     // Error: Incorrect formula for rectangle area.
247     // Solution: return length * width;
248 }
249
250 int square(int n)
251 {
252     return n * n;
253 }
254
255 int cube(int n)
256 {
257     return n * n;
258     // Error: Incorrect formula for cube.
259     // Solution: return n * n * n;
260 }
```